



Advanced Destructuring & Syntax

Topics: Nested Destructuring, Computed Property Names, Variable Swapping, Function Parameter Destructuring, Logical Assignment Operators, Optional Catch Binding

1. Advanced Destructuring কী?

Destructuring হলো object বা array থেকে data বের করে সরাসরি variable-এ assign করার syntax ।

Basic example:

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```

Advanced destructuring-এ আমরা আরও complex structure handle করতে পারি:

```
Object  
├─ Object  
│   └─ Object  
└─ Array  
    └─ Object
```

এছাড়াও আমরা:

- nested values extract করতে পারি
 - computed property ব্যবহার করতে পারি
 - variables swap করতে পারি
 - function parameters destructure করতে পারি
 - default values দিতে পারি
 - rename করতে পারি
-

2. Nested Destructuring

যখন object-এর ভিতরে object থাকে, তখন তাকে nested destructuring বলা হয়।

Example:

```
const user = {  
  name: "Sharlar",  
  address: {  
    city: "Rangpur",  
    country: "Bangladesh"  
  }  
};
```

Normal way:

```
const city = user.address.city;  
const country = user.address.country;
```

Nested destructuring:

```
const {  
  address: { city, country }  
} = user;  
  
console.log(city);  
console.log(country);
```

Output:

```
Rangpur  
Bangladesh
```

3. Nested Object Destructuring Mental Model

এই object:

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Rangpur",  
    country: "Bangladesh"  
  }  
};
```

Structure:

```
user  
├── name  
└── address  
    ├── city  
    └── country
```

Destructuring:

```
const {  
  address: { city }  
} = user;
```

মানে:

```
user  
↓  
address  
↓  
city  
↓  
city variable
```

4. Nested Array Destructuring

Array-এর ভিতরে array থাকলেও destructuring করা যায়।

```
const numbers = [10, [20, 30]];

const [first, [second, third]] = numbers;

console.log(first);
console.log(second);
console.log(third);
```

Output:

```
10
20
30
```

5. Mixed Nested Destructuring

Object-এর মধ্যে array:

```
const user = {
  name: "Shariar",
  skills: ["JavaScript", "React", "Next.js"]
};
```

Destructure:

```
const {
  name,
  skills: [firstSkill, secondSkill]
} = user;

console.log(name);
console.log(firstSkill);
console.log(secondSkill);
```

Output:

```
Shariar  
JavaScript  
React
```

6. Array-এর মধ্যে Object

```
const users = [  
  {  
    name: "Shariar",  
    age: 31  
  },  
  {  
    name: "Rahim",  
    age: 28  
  }  
];
```

Destructuring:

```
const [  
  { name: firstName },  
  { name: secondName }  
] = users;  
  
console.log(firstName);  
console.log(secondName);
```

Output:

```
Shariar  
Rahim
```

7. Deep Nested Destructuring

Complex API response-এর মধ্যে structure:

```
const response = {
  data: {
    user: {
      profile: {
        name: "Shariar",
        location: {
          city: "Rangpur"
        }
      }
    }
  }
};
```

Destructure:

```
const {
  data: {
    user: {
      profile: {
        name,
        location: { city }
      }
    }
  }
} = response;

console.log(name);
console.log(city);
```

Output:

```
Shariar
Rangpur
```

কিন্তু সাবধান

Deep destructuring code অনেক complex হয়ে যেতে পারে।

সবসময় deepest level পর্যন্ত destructure করা প্রয়োজন নেই।

8. Nested Destructuring + Rename

```
const user = {
  profile: {
    name: "Sharlar"
  }
};

const {
  profile: { name: userName }
} = user;

console.log(userName);
```

এখানে:

```
name
↓
userName
```

9. Nested Destructuring + Default Value

```
const user = {
  name: "Sharlar",
  address: {}
};

const {
  address: {
    city = "Unknown"
  }
} = user;

console.log(city);
```

Output:

```
Unknown
```

10. Missing Nested Object-এর সমস্যা

এটা:

```
const user = {  
  name: "Sharlar"  
};  
  
const {  
  address: { city }  
} = user;
```

Error হবে।

কারণ:

```
user.address  
  ↓  
undefined
```

তারপর:

```
undefined.city
```

access করার চেষ্টা হচ্ছে।

11. Nested Destructuring-এর Safe Default

এভাবে default দিতে পারো:

```
const user = {  
  name: "Sharlar"  
};  
  
const {  
  address: {  
    city = "Unknown"  
  } = {}  
} = user;
```

```
console.log(city);
```

এখানে:

```
address = {}
```

হিসেবে fallback দেওয়া হয়েছে।

12. Destructuring-এর Default Value কখন কাজ করে?

Default value তখনই কাজ করে যখন value `undefined`।

```
const { name = "Guest" } = {  
  name: undefined  
};  
  
console.log(name);
```

Output:

```
Guest
```

কিন্তু:

```
const { name = "Guest" } = {  
  name: null  
};  
  
console.log(name);
```

Output:

```
null
```

কারণ `null` `undefined` নয়।

13. Computed Property Names in Destructuring

Computed property name ব্যবহার করে dynamic property name destructure করা যায়।

ধরো:

```
const key = "name";

const user = {
  name: "Shariar",
  age: 31
};
```

এখন:

```
const {
  [key]: value
} = user;

console.log(value);
```

Output:

```
Shariar
```

14. Computed Destructuring Mental Model

এখানে:

```
const key = "name";

const {
  [key]: value
} = user;
```

প্রথমে:

key

এর value:

"name"

তাই effectively:

```
const {  
  name: value  
} = user;
```

হয়ে যায়।

15. Dynamic Property Access

Computed destructuring useful যখন property name আগে থেকে জানা নেই।

```
const field = "email";  
  
const user = {  
  name: "Sharlar",  
  email: "sharlar@example.com"  
};  
  
const {  
  [field]: value  
} = user;  
  
console.log(value);
```

Output:

sharlar@example.com

16. Computed Property + Rename

Computed destructuring-এর syntax:

```
const {  
  [dynamicKey]: localVariable  
} = object;
```

Example:

```
const key = "age";  
  
const user = {  
  age: 31  
};  
  
const {  
  [key]: userAge  
} = user;  
  
console.log(userAge);
```

17. Computed Property + Default Value

```
const key = "role";  
  
const user = {  
  name: "Sharir"  
};  
  
const {  
  [key]: role = "user"  
} = user;  
  
console.log(role);
```

Output:

```
user
```

18. Variable Swapping with Destructuring

Traditional JavaScript:

```
let a = 10;  
let b = 20;  
  
const temp = a;  
  
a = b;  
b = temp;
```

Destructuring দিয়ে খুব সহজ:

```
let a = 10;  
let b = 20;  
  
[a, b] = [b, a];  
  
console.log(a);  
console.log(b);
```

Output:

```
20  
10
```

19. কীভাবে Swap কাজ করে?

```
[a, b] = [b, a];
```

Right side আগে evaluate হয়:

```
[b, a]
```

যদি:

```
a = 10  
b = 20
```

তাহলে temporary array conceptually:

```
[20, 10]
```

তারপর:

```
a ← 20  
b ← 10
```

20. তিনটি Variable Swap

```
let a = 1;  
let b = 2;  
let c = 3;  
  
[a, b, c] = [c, a, b];  
  
console.log(a);  
console.log(b);  
console.log(c);
```

Output:

```
3  
1  
2
```

21. Swap করার সময় `const` ব্যবহার করা যাবে?

না, যদি variable-এর value পরিবর্তন করতে চাও।

এটা:

```
const a = 10;
const b = 20;

[a, b] = [b, a];
```

error করবে।

কারণ `const` variable reassign করা যায় না।

Swap-এর ক্ষেত্রে:

```
let a = 10;
let b = 20;
```

ব্যবহার করতে হবে।

22. Function Parameter Destructuring

Function parameter-এর মধ্যেই destructuring করা যায়।

Normal:

```
function greet(user) {
  console.log(user.name);
  console.log(user.age);
}
```

Destructuring:

```
function greet({ name, age }) {
  console.log(name);
  console.log(age);
}
```

Call:

```
greet({
  name: "Shariar",
```

```
    age: 31
  });
```

23. কেন Parameter Destructuring Useful?

ধরো React component:

```
function UserCard(props) {
  return `
    ${props.name}
    ${props.email}
  `;
}
```

Destructuring:

```
function UserCard({ name, email }) {
  return `
    ${name}
    ${email}
  `;
}
```

Code cleaner হয়।

24. Parameter Destructuring + Default Values

```
function greet({
  name = "Guest",
  age = 0
}) {
  console.log(name, age);
}
```

Call:

```
greet({});
```

Output:

Guest 0

25. Parameter Destructuring + Rename

```
function greet({
  name: userName,
  age: userAge
}) {
  console.log(userName);
  console.log(userAge);
}
```

এখানে:

```
name → userName
age  → userAge
```

26. Nested Parameter Destructuring

```
function printCity({
  profile: {
    address: {
      city
    }
  }
}) {
  console.log(city);
}
```

Call:

```
printCity({
  profile: {
    address: {
      city: "Rangpur"
    }
  }
})
```

```
    }  
  }  
});
```

27. Parameter Destructuring + Optional Object

একটি important problem:

```
function greet({ name }) {  
  console.log(name);  
}  
  
greet();
```

এটা error করবে।

কারণ parameter-এর value:

```
undefined
```

28. Safe Parameter Destructuring

Default object দিতে পারো:

```
function greet({ name } = {}) {  
  console.log(name);  
}  
  
greet();
```

এখন error হবে না।

আর default value:

```
function greet({  
  name = "Guest"  
} = {}) {
```

```
    console.log(name);  
  }
```

29. Array Parameter Destructuring

Function parameter-এ array-ও destructure করা যায়।

```
function sum([a, b]) {  
  return a + b;  
}  
  
console.log(sum([10, 20]));
```

Output:

```
30
```

30. Rest Pattern in Parameter Destructuring

```
function processUser({  
  name,  
  ...otherInfo  
) {  
  console.log(name);  
  console.log(otherInfo);  
}
```

Call:

```
processUser({  
  name: "Shariar",  
  age: 31,  
  city: "Rangpur"  
});
```

`otherInfo`:

```
{  
  age: 31,  
  city: "Rangpur"  
}
```

31. Logical Assignment Operators

JavaScript-এ তিনটি important logical assignment operator:

```
??=  
||=  
&&=
```

এগুলো combination:

```
logical operator  
  +  
assignment
```

32. `||=` Operator

Syntax:

```
x ||= value;
```

Conceptually:

```
x || (x = value);
```

যদি `x` falsy হয়, তাহলে value assign হবে।

33. `||=` Example

```
let username = "";  
  
username ||= "Guest";  
  
console.log(username);
```

Output:

Guest

কারণ:

`""` → falsy

34. `||=` Truthy/Falsy Based

```
let value = 0;  
  
value ||= 100;  
  
console.log(value);
```

Output:

100

কারণ `0` falsy।

আর:

```
let value = 50;  
  
value ||= 100;
```

```
console.log(value);
```

Output:

```
50
```

কারণ `50` truthy।

35. `||=` এবং Falsy Values

`||=` নিচের values-কে fallback হিসেবে treat করবে:

```
false  
0  
""  
NaN  
null  
undefined
```

তাই যদি `0` valid value হয়, `||=` dangerous হতে পারে।

36. `??=` Operator

Syntax:

```
x ??= value;
```

Conceptually:

```
x ?? (x = value);
```

কিন্তু `??` শুধুমাত্র:

```
null  
undefined
```

কে missing হিসেবে treat করে।

37. ??= Example

```
let age = 0;  
  
age ??= 18;  
  
console.log(age);
```

Output:

0

কারণ:

```
0 !== null  
0 !== undefined
```

তাই assignment হবে না।

38. ??= বনাম ||=

এটা খুব important।

||=

```
let value = 0;  
  
value ||= 100;  
  
console.log(value);
```

Output:

100

??=

```
let value = 0;  
  
value ??= 100;  
  
console.log(value);
```

Output:

0

39. কখন `||=` ব্যবহার করব?

যখন:

"Value falsy হলে fallback দাও"

Example:

```
let username = "";  
  
username ||= "Guest";
```

40. কখন `??=` ব্যবহার করব?

যখন:

"Value শুধু null বা undefined হলে fallback দাও"

Example:

```
let count = 0;  
  
count ??= 10;
```

0 preserve থাকবে।

Configuration/default values-এর ক্ষেত্রে `??=` অনেক useful।

41. `&&=` Operator

Syntax:

```
x &&= value;
```

Conceptually:

```
x && (x = value);
```

যদি `x` truthy হয়, তাহলে নতুন value assign হবে।

42. `&&=` Example

```
let isLoggedIn = true;

isLoggedIn &&= false;

console.log(isLoggedIn);
```

Output:

```
false
```

কারণ initial value `true` ছিল।

আর:

```
let isLoggedIn = false;

isLoggedIn &&= true;

console.log(isLoggedIn);
```

Output:

false

কারণ initial value falsy, তাই assignment হয়নি।

43. তিনটি Logical Assignment একসাথে

Operator	Assignment হয় যখন
<code> =</code>	Left side falsy
<code>??=</code>	Left side <code>null</code> / <code>undefined</code>
<code>&&=</code>	Left side truthy

Mental model:

```
x ||= y
→ x falsy হলে y

x ??= y
→ x null/undefined হলে y

x &&= y
→ x truthy হলে y
```

44. Object-এর সাথে Logical Assignment

```
const user = {
  name: "",
  age: 0,
  role: null
};

user.name ||= "Guest";
user.age ??= 18;
user.role ??= "user";
```

```
console.log(user);
```

Result conceptually:

```
{
  name: "Guest",
  age: 0,
  role: "user"
}
```

45. Nested Property-এর সাথে `??=`

```
const config = {};

config.options ??= {};

config.options.theme ??= "dark";

console.log(config);
```

Result:

```
{
  options: {
    theme: "dark"
  }
}
```

এটা configuration initialization-এর জন্য useful ।

46. Logical Assignment Short-Circuit করে

Logical assignment operators unnecessary assignment avoid করতে পারে ।

Example:

```
let value = 10;

value ||= 20;
```

যেহেতু `value` truthy, `20` assign হবে না।

একইভাবে:

```
let value = null;

value ??= 20;
```

এখানে assignment হবে।

47. Getter/Setter-এর সাথে Logical Assignment

Logical assignment শুধু simple variables-এর জন্য নয়; object properties-এর getter/setter-এর সাথেও কাজ করতে পারে।

Example:

```
const user = {
  _name: "",

  get name() {
    return this._name;
  },

  set name(value) {
    this._name = value;
  }
};

user.name ||= "Guest";

console.log(user.name);
```

এখানে property access এবং assignment logical condition অনুযায়ী evaluate হয়।

48. Optional Catch Binding

আপে `catch` লিখলে সাধারণত error parameter দিতে হতো:

```
try {  
  JSON.parse("invalid");  
} catch (error) {  
  console.log("Something went wrong");  
}
```

কিন্তু যদি error object দরকারই না হয়?

Modern JavaScript-এ:

```
try {  
  JSON.parse("invalid");  
} catch {  
  console.log("Something went wrong");  
}
```

এটাকে **optional catch binding** বলা হয়।

49. কেন Optional Catch Binding দরকার?

যদি:

```
catch (error)
```

এর `error` ব্যবহার না করো, তাহলে unnecessary variable।

Instead:

```
catch {  
}
```

cleaner।

50. Catch Binding কখন ব্যবহার করবে?

যখন error-এর details দরকার নেই।

```
try {
  doSomething();
} catch {
  showFallback();
}
```

কিন্তু যদি debugging/logging দরকার হয়:

```
try {
  doSomething();
} catch (error) {
  console.error(error);
}
```

তাহলে parameter ব্যবহার করাই উচিত।

51. Optional Catch Binding + Finally

```
try {
  doSomething();
} catch {
  console.log("Failed");
} finally {
  console.log("Cleanup");
}
```

`catch` parameter optional হলেও `finally` আগের মতোই কাজ করে।

52. Real-World API Example

ধরো API response:

```
const response = {
  data: {
```

```

    user: {
      profile: {
        name: "Shariar",
        location: {
          city: "Rangpur",
          country: "Bangladesh"
        }
      }
    }
  }
};

```

এখন:

```

const {
  data: {
    user: {
      profile: {
        name,
        location: { city, country }
      }
    }
  }
} = response;

console.log(name);
console.log(city);
console.log(country);

```

এই pattern API response handling-এ দেখা যায়।

53. React Component Example

React component:

```

function UserCard({
  name,
  email,
  address: {
    city
  }
}) {
  return (

```

```

    <div>
      <h2>{name}</h2>
      <p>{email}</p>
      <p>{city}</p>
    </div>
  );
}

```

Usage:

```

<UserCard
  name="Shariar"
  email="shariar@example.com"
  address={{
    city: "Rangpur"
  }}
/>

```

এখানে props-এর মধ্যেই destructuring হয়েছে।

54. React-এর Default Props Pattern

```

function Button({
  text = "Click",
  disabled = false
}) {
  return (
    <button disabled={disabled}>
      {text}
    </button>
  );
}

```

এতে caller `text` না দিলে:

Click

ব্যবহার হবে।

55. Function Parameter Destructuring Best Practice

যদি function অনেক properties নেয়:

Bad:

```
function createUser(  
  name,  
  age,  
  email,  
  city,  
  country,  
  role  
) {}
```

Better:

```
function createUser({  
  name,  
  age,  
  email,  
  city,  
  country,  
  role  
}) {}
```

Call:

```
createUser({  
  name: "Shariar",  
  age: 31,  
  email: "shariar@example.com",  
  city: "Rangpur",  
  country: "Bangladesh",  
  role: "developer"  
});
```

এটা readability এবং maintainability বাড়ায়।

56. Destructuring vs Optional Chaining

দুটো একই জিনিস নয়।

Optional chaining:

```
const city = user?.address?.city;
```

Destructuring:

```
const {  
  address: { city }  
} = user;
```

যদি nested property missing হওয়ার possibility থাকে, destructuring-এ default structure দিতে হতে পারে।

57. Destructuring vs Property Access

Simple case:

```
console.log(user.name);
```

Destructuring:

```
const { name } = user;  
  
console.log(name);
```

যদি value একবারই ব্যবহার করো, direct property access অনেক সময় simpler।

যদি multiple properties repeatedly use করো, destructuring cleaner হতে পারে।

58. Over-Destructuring Avoid করো

এমন code:

```
const {  
  a: {  
    b: {  
      c: {  
        d: {  
          e  
        }  
      }  
    }  
  }  
} = data;
```

Technically valid হলেও maintain করা কঠিন।

Better:

```
const profile = data.a.b.c.d;  
  
const { e } = profile;
```

Readability priority।

59. Logical Assignment — Common Mistake

ধরো:

```
let count = 0;  
  
count ||= 10;
```

অনেকে ভাবতে পারে `count` already আছে তাই `10` assign হবে না।

কিন্তু:

```
0 → falsy
```

তাই:

```
count = 10
```

হয়ে যাবে।

যদি `0` valid value হয়:

```
count ??= 10;
```

ব্যবহার করো।

60. `||=` vs `??=` — Practical Rule

এই দুইটা মনে রাখো:

```
||=
→ falsy check

??=
→ null/undefined check
```

Example:

```
let name = "";
name ||= "Guest";
```

Use `||=`।

আর:

```
let count = 0;
count ??= 10;
```

Use `??=`।

61. `&&=` — Practical Use Case

ধরো:

```
const user = {  
  isAdmin: true  
};  
  
user.isAdmin &&= false;
```

এটা effectively বলছে:

যদি `isAdmin` সত্য হয়, তাহলে `false` করে দাও।

আর:

```
const user = {  
  isAdmin: false  
};  
  
user.isAdmin &&= false;
```

কোনো assignment হবে না।

62. Advanced Syntax — একসাথে Mental Model

এই section-এর syntaxগুলো:

```
Destructuring  
|  
├─ Nested  
├─ Computed property  
├─ Swap  
└─ Function parameters  
    |  
    ↓  
Logical Assignment  
|  
├─ ||=  
├─ ??=  
└─ &&=  
    |  
    ↓  
Error Handling Syntax
```

```
|  
└─ catch {}
```

63. Output Prediction — 1

```
let a = 10;  
let b = 20;  
  
[a, b] = [b, a];  
  
console.log(a, b);
```

Output:

```
20 10
```

64. Output Prediction — 2

```
let value = 0;  
  
value ||= 100;  
  
console.log(value);
```

Output:

```
100
```

65. Output Prediction — 3

```
let value = 0;  
  
value ??= 100;
```

```
console.log(value);
```

Output:

```
0
```

66. Output Prediction — 4

```
let value = "Hello";  
  
value &&= "World";  
  
console.log(value);
```

Output:

```
World
```

67. Output Prediction — 5

```
const key = "name";  
  
const user = {  
  name: "Sharar"  
};  
  
const {  
  [key]: value  
} = user;  
  
console.log(value);
```

Output:

Shariar

68. Output Prediction — 6

```
function greet({  
  name = "Guest"  
} = {}) {  
  console.log(name);  
}  
  
greet();
```

Output:

Guest

69. Practice — Easy

Problem 1

এই object থেকে nested destructuring ব্যবহার করে `city` বের করো:

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Rangpur",  
    country: "Bangladesh"  
  }  
};
```

Expected:

Rangpur

Problem 2

দুটি variable swap করো:

```
let x = 100;  
let y = 200;
```

শুধুমাত্র destructuring ব্যবহার করে।

Problem 3

এই function-টাকে parameter destructuring দিয়ে rewrite করো:

```
function printUser(user) {  
  console.log(user.name);  
  console.log(user.email);  
}
```

70. Practice — Medium

Problem 4

Dynamic property name ব্যবহার করো:

```
const key = "email";  
  
const user = {  
  name: "Sharir",  
  email: "hello@example.com"  
};
```

`email` value বের করো।

Problem 5

এমন function বানাও:

```
getUserInfo({  
  name: "Sharir",  
  age: 31  
});
```

যেখানে `name` এবং `age` destructuring-এর মাধ্যমে পাওয়া যাবে।

যদি কোনো value না থাকে:

```
name → Guest  
age → 0
```

Problem 6

এই code-এর behavior explain করো:

```
let score = 0;  
  
score ||= 50;  
  
console.log(score);
```

তারপর `??=` ব্যবহার করলে কী output হবে দেখাও।

71. Practice — Hard

Problem 7

এই API response:

```
const response = {  
  data: {  
    user: {  
      profile: {  
        name: "Sharir",  
        contact: {  
          email: "hello@example.com"  
        }  
      }  
    }  
  }  
}
```

```
}  
}  
};
```

এক statement-এ destructuring করে বের করো:

```
name  
email
```

Problem 8

এই function:

```
function createUser(options) {  
  // ...  
}
```

এটাকে এমনভাবে design করো যেন:

```
createUser({  
  name: "Sharar",  
  role: "developer"  
});
```

এবং default থাকে:

```
role → user  
age → 0
```

Problem 9

একটি config object বানাও যেখানে:

```
theme  
language  
pageSize
```

??= ব্যবহার করে default values initialize করো।

এমনভাবে করো যেন:

```
pageSize = 0
```

থাকলে সেটা পরিবর্তন না হয়।

72. Real-World Challenge

একটি user configuration:

```
const config = {
  user: {
    name: "Sharlar",
    preferences: {
      theme: null,
      language: "bn"
    }
  }
};
```

Requirements:

1. Nested destructuring ব্যবহার করে `name` বের করো।
2. `theme` null হলে `"dark"` set করো।
3. `language` existing থাকলে preserve করো।
4. `name` না থাকলে `"Guest"` fallback দাও।
5. Function parameter destructuring ব্যবহার করে পুরো operation-টি একটি function-এর মধ্যে রাখো।

73. Interview Questions

Beginner

1. Destructuring কী?
2. Nested destructuring কী?
3. Array এবং object destructuring-এর difference কী?
4. Function parameter destructuring কী?
5. Variable swap কীভাবে destructuring দিয়ে করা যায়?
6. Computed property name কী?

Intermediate

1. `||=` কী?
2. `??=` কী?
3. `&&=` কী?
4. `||=` এবং `??=` -এর difference কী?
5. `0` থাকলে `||=` কেন unexpected result দিতে পারে?
6. Default destructuring value কখন কাজ করে?
7. Nested destructuring-এ missing object হলে কী হয়?
8. `catch {}` কী?
9. Optional catch binding কী?

Advanced

1. Computed property destructuring কীভাবে কাজ করে?
2. `||=` কীভাবে short-circuit করে?
3. `??=` এবং `||=` -এর semantic difference কী?
4. Logical assignment-এর সাথে getter/setter কীভাবে interact করে?
5. Deep destructuring কখন avoid করা উচিত?
6. Function parameter destructuring-এর default object কেন দরকার হতে পারে?
7. Destructuring এবং optional chaining-এর difference কী?
8. API response handling-এ destructuring-এর সুবিধা ও downside কী?

74. Quick Revision

Nested Destructuring

```
const {  
  address: { city }  
} = user;
```

Computed Property

```
const key = "name";  
  
const {  
  [key]: value  
} = user;
```

Swap

```
[a, b] = [b, a];
```

Function Parameter Destructuring

```
function greet({ name, age }) {  
  console.log(name, age);  
}
```

Parameter Default

```
function greet({  
  name = "Guest"  
} = {}) {  
  console.log(name);  
}
```

||=

```
value ||= fallback;
```

Falsy হলে assign করবে।

??=

```
value ??= fallback;
```

null বা undefined হলে assign করবে।

&&=

```
value &&= newValue;
```

Truthy হলে assign করবে।

Optional Catch Binding

```
try {  
  doSomething();  
} catch {  
  handleError();  
}
```

Error object দরকার না হলে।

75. Final Cheat Sheet

Feature	Meaning
<code>{ a } = obj</code>	Object destructuring
<code>[a] = arr</code>	Array destructuring
<code>{ a: b }</code>	Rename
<code>{ a = 10 }</code>	Default
<code>{ a: { b } }</code>	Nested destructuring
<code>{ [key]: value }</code>	Computed property
<code>[a, b] = [b, a]</code>	Swap
<code>function f({ a })</code>	Parameter destructuring
<code>x = y</code>	Falsy → assign
<code>x ??= y</code>	Null/undefined → assign
<code>x &&= y</code>	Truthy → assign
<code>catch {}</code>	Ignore catch parameter

Mastery Checklist

- [] Nested object destructuring বুঝি
- [] Nested array destructuring বুঝি

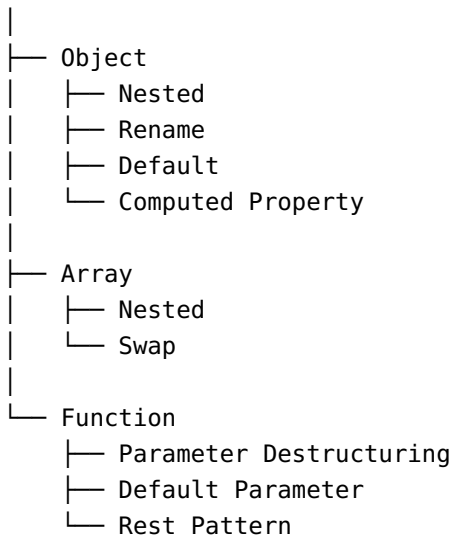
- [] Mixed object/array destructuring পারি
- [] Nested destructuring + rename পারি
- [] Nested destructuring + default পারি
- [] Missing nested object handle করতে পারি
- [] Computed property destructuring বুঝি
- [] Dynamic key দিয়ে property extract করতে পারি
- [] Destructuring দিয়ে variables swap করতে পারি
- [] Function parameter destructuring পারি
- [] Parameter destructuring-এর default object বুঝি
- [] Array parameter destructuring পারি
- [] Rest pattern-এর সাথে destructuring পারি
- [] `||=` বুঝি
- [] `??=` বুঝি
- [] `&&=` বুঝি
- [] `||=` বনাম `??=` explain করতে পারি
- [] `0`, `false`, `""` কেন `||=` -এ সমস্যা করতে পারে বুঝি
- [] Optional catch binding বুঝি
- [] Deep destructuring কখন avoid করতে হবে জানি
- [] React props destructuring confidently করতে পারি
- [] API response destructuring করতে পারি



Final Mental Model

এই পুরো section-টা মনে রাখার সবচেয়ে সহজ উপায়:

Advanced Destructuring



Logical Assignment

- |
- └─ ||= → falsy হলে assign
- └─ ??= → null/undefined হলে assign
- └─ &&= → truthy হলে assign

Error Handling

- |
- └─ catch {}
→ error parameter প্রয়োজন না হলে

★ সবচেয়ে গুরুত্বপূর্ণ Rule

||= → "Falsy হলে fallback দাও"

??= → "শুধু null/undefined হলে fallback দাও"

&&= → "Truthy হলে নতুন value দাও"

আর destructuring-এর জন্য:

Object → {}
Array → []

Nested → structure follow করো

Rename → :
Default → =

Computed → [expression]

এই conceptsগুলো ঠিকমতো আয়ত্ত করলে modern JavaScript-এর অনেক code অনেক বেশি **short, readable** এবং **expressive** মনে হবে।